



X Monitor

Automated Post Detection System

Architecture & Process Guide — For Non-Technical Readers

June 2026

10
Users Tracked

45 sec
Update Cycle

24/7
Running

0
Duplicates

SECTION 1 | What is X Monitor?

Simple Answer: X Monitor is a software robot that keeps watching selected Twitter/X accounts 24 hours a day, 7 days a week. The moment any watched person posts something new, you get an instant notification on your Windows computer — without you having to check Twitter manually.

Real-World Analogy

Imagine you hired a personal assistant whose only job is to refresh Elon Musk's Twitter page every few seconds and immediately call you when he posts something new. X Monitor does exactly that — but it's software, so it never sleeps, never gets tired, and never misses a post.

What It Does vs. What It Does NOT Do

■ DOES	Watches public Twitter/X profiles silently
■ DOES	Sends you a Windows desktop notification when a new post appears
■ DOES	Shows a web dashboard at http://localhost:8080
■ DOES	Remembers what it already notified you about (no duplicates)
■ DOES	Runs continuously in the background on your computer
■ DOES NOT	Like, comment, repost, or take any action on Twitter
■ DOES NOT	Require you to log into Twitter
■ DOES NOT	Use Twitter's official API (reads public pages like a browser)
■ DOES NOT	Send your data anywhere outside your computer
■ DOES NOT	Notify you twice for the same post

SECTION 2 | The Big Picture — How It's Built

The system has **three main engines** running simultaneously inside your computer. Think of them as three workers doing different jobs at the same time.

■ Engine 1

The Monitoring Brain

Opens Twitter/X in a hidden browser window and reads posts. Like someone silently browsing Twitter but never interacting with it.

■ Engine 2

The Memory System

Stores what posts it has already seen in a database file on your computer. This is why you never get duplicate notifications.

■ Engine 3

The Dashboard

Runs a mini website at <http://localhost:8080> so you can see live statistics, all detected posts, and manage settings from your browser.

How The Three Engines Talk To Each Other

Step	Engine	What Happens
1	Monitoring Brain	Opens x.com/username in hidden browser, reads the page
2	Monitoring Brain	Finds the newest post link on the page
3	Memory System	Checks: have we seen this post before?
4	Memory System	YES → Do nothing. NO → It's new!
5	Monitoring Brain	Sends Windows notification to you
6	Memory System	Saves the new post ID so it won't notify again
7	Dashboard	Updates the web page with the new detection

SECTION 3 | Step-by-Step Workflow

Here is exactly what happens from the moment you start the program to the moment you receive a notification:

START	You run: <code>python main.py</code> You type this command once. Everything else is automatic.
0 sec	Program wakes up Opens a hidden Chrome browser window (you won't see it). Connects to the database file. Starts the web dashboard server.
2 sec	Workers begin 3 workers start simultaneously. Each worker gets assigned about 8-9 usernames to monitor. They start checking accounts one by one.
~8 sec	First account checked Worker opens x.com/elonmusk , reads the page, finds the latest post. Since it's the FIRST TIME, it just saves the post ID. No notification yet. (Reason: we don't know if this post is new or old.)
	All 10 accounts seeded All 10 accounts have been checked for the first time. The system now knows the 'last seen' post for everyone.
~90 sec	Second cycle begins Workers start checking again. This time, they compare the new post ID with the stored one. Same ID = nothing happened. Different ID = NEW POST!
	Elon Musk posts something Workers are checking continuously. The next time a worker checks @elonmusk (within ~45 seconds), it finds a different post ID.
NOTIFY	You get the notification A popup appears on your Windows screen: 'New post from Elon Musk — @elonmusk — Click to open' The web dashboard also updates automatically within 3 seconds.

SECTION 4 | How Long Does It Take to Detect a New Post?

Bottom Line: On average, you will receive a notification within **30 to 60 seconds** of someone posting. In the best case it could be as fast as 8 seconds. In the worst case, up to 2 minutes.

Why Can't It Be Instant?

Opening a web page takes time. Just like when YOU open Twitter in your browser it takes 3-8 seconds to load — the same happens here. The program has to actually open the person's profile page and read it. It cannot be faster than a normal browser page load.

Scenario	Time	Why
Best case — worker was just about to check this user	8-15 sec 8-15 sec	Page loads in ~8 seconds
Average — worker is mid-cycle through other users	30-60 sec 30-60 sec	Has to finish checking other users first
Worst case — worker just started a new cycle	60-120 sec	Worker at the beginning of its 8-user list
If internet is slow or Twitter is slow	Up to 3 min	Page load timeout + retry logic kicks in

The 45-Second Cycle Explained

Every 45 seconds is one full 'cycle'. In each cycle, every monitored user gets checked exactly once. The 3 workers divide the users between them so the whole cycle completes within 45 seconds. If a post happens at second 0 of a cycle: best case = 8 sec detection
If a post happens at second 44 of a cycle: worst case = $44 + 8 = 52$ sec

SECTION 5 | How Will I Know When a Post Is Detected?

You will be notified in **TWO** ways simultaneously:

■ Method 1

Windows Desktop Toast Notification

A popup appears in the bottom-right corner of your Windows screen (same as WhatsApp or email notifications). It shows: • The person's display name (e.g. 'Elon Musk') • Their Twitter handle (e.g. '@elonmusk') • The exact time it was detected • A button: 'Open Post' — clicking this opens the post in your browser

■ Method 2

Web Dashboard (localhost:8080)

Open your browser and go to <http://localhost:8080>. You will see: • 'Recent Detections' feed — every new post with a clickable link • Live stats (total checks, new posts found, errors) • Status of all 10 monitored users (last checked time) • The dashboard refreshes itself every 3 seconds automatically

Important: The program must be running on your computer for notifications to work. If you close the terminal window or shut down your computer, monitoring stops. When you restart it, it picks up from where it left off — no duplicate notifications for old posts.

SECTION 6 | System Components — Plain English

<code>config.py</code>	The Settings Panel Contains all the adjustable settings: which users to monitor, how often to check, how many workers to use. Changing this file changes the behavior of the entire system. No code knowledge needed to add new users.
<code>database.py</code>	The Memory System Uses a file called <code>monitor.db</code> to remember the last post seen for each user. This prevents duplicate notifications. Even if the program restarts, it remembers what it already notified you about.
<code>browser.py</code>	The Browser Manager Controls one hidden Chrome browser window. Shares this one browser among all workers (instead of opening 10 browsers) to save RAM and CPU. Auto-restarts the browser if it crashes.
<code>monitor.py</code>	The Twitter Reader The most important component. Opens a Twitter profile page, reads all the post links, and finds the newest one. Handles cases like login popups, slow loading, and suspended accounts.
<code>scheduler.py</code>	The Traffic Controller Divides 25 users among 3 workers. Makes sure workers don't duplicate each other's work. Handles errors gracefully — if one check fails, the others keep going. Automatically restarts crashed workers.
<code>notifier.py</code>	The Alarm System Sends the Windows desktop popup notification. The notification includes the person's name, handle, detection time, and a button to open the post.
<code>web.py</code>	The Dashboard Website Runs a mini website at <code>localhost:8080</code> showing live stats, recent detections, user statuses, and settings. Updates every 3 seconds automatically.
<code>ui_state.py</code>	The Communication Bridge A shared memory space that lets the monitoring engine and the web dashboard talk to each other without interfering. Like a whiteboard that both can read and write on.

SECTION 7 | Frequently Asked Questions

Q1. How is this different from just following someone on Twitter?

When you follow someone on Twitter, you see their posts in your feed — but only when you manually open Twitter. X Monitor notifies you immediately on your desktop, even if Twitter is not open. You don't need a Twitter account.

Q2. Does this require me to log into Twitter?

No. The system reads only public Twitter profiles — the same pages anyone can see in a browser without logging in. No Twitter credentials are needed.

Q3. Will Twitter block or ban my account?

There is no account involved. The system reads public pages like any visitor would. Twitter may occasionally slow down or challenge the requests (which the system handles by retrying), but no account can be banned.

Q4. What if someone posts while the program is off?

You will miss that post. The program only detects new posts while it is running. When you restart it, it will use the latest post as the new baseline — it will NOT send notifications for posts made while it was offline.

Q5. Can it monitor private (protected) Twitter accounts?

No. It can only read public profiles — the same ones anyone can see without logging in. Protected/private accounts are not visible to the system.

Q6. How much does it cost to run?

Nothing. It uses no paid APIs or subscriptions. It runs entirely on your computer using free, open-source software. The only cost is electricity.

Q7. Will it slow down my computer?

Very little. It uses one hidden browser and 3 workers. RAM usage is about 150-200 MB (similar to having one Chrome tab open). CPU usage is less than 1% when idle and spikes briefly to 5-10% during page loads.

Q8. What happens if the internet goes down?

The system automatically retries each failed check up to 2 times. If the internet is still down, it logs an error and moves on to the next user. When the internet comes back, it resumes normally.

Q9. Can I add or remove users without restarting?

Yes. Open the web dashboard at localhost:8080, go to Settings, edit the user list, and click Save. Changes take effect on the next monitoring cycle.

Q10. What if the same person posts 10 times in a row?

You will receive 10 separate notifications — one for each new post detected. However, you will NEVER receive the same notification twice for the same post.

SECTION 7 | Frequently Asked Questions (continued)

Q11. How accurate is the 45-second detection time?

Very accurate under normal conditions. The 45 seconds is the maximum cycle time. In practice, most detections happen within 30-60 seconds. Network slowness or Twitter loading issues can occasionally extend this to 2 minutes.

Q12. What if I close the terminal/command window?

The program stops. All monitoring pauses. No notifications will be sent. Restart by running 'python main.py' again. The database remembers where it left off.

Q13. Where are the notifications stored? Can I see old ones?

Yes. Open the web dashboard (localhost:8080) → Dashboard tab. The 'Recent Detections' section shows the last 50 notifications with clickable links.

Q14. Can I get notifications on my phone?

Currently, notifications only appear as Windows desktop popups. Phone notifications would require additional integration (e.g., WhatsApp API or push notification service) which is not part of this version.

Q15. What if Twitter changes its website design?

The system may temporarily stop detecting posts. It would need to be updated to match the new page structure. This is a known limitation of web-scraping based systems. Twitter changes its layout occasionally.

Q16. Is my data safe? Does it send anything to the internet?

All data stays on your computer. The system only makes outgoing requests TO Twitter (to read public pages). No data is sent to any third-party server. The web dashboard only works on your local network (localhost).

Q17. Can multiple people use this on different computers?

Each person would need to run it on their own computer. However, if you share your localhost:8080 on a local network, others on the same WiFi could view the dashboard (but not receive Windows notifications).

Q18. How many users can it monitor at once?

Practically, 20-30 users works well. More than 50 users would require increasing the check interval or worker count to ensure all users are checked within the cycle time. The config file makes this easy to adjust.

Q19. What does 'seeding' mean in the logs?

'Seeding' happens the first time a user is checked. Instead of sending a notification (since we don't know if the post is new or old), it just saves the current post ID as the baseline. From the next check onwards, any change in post ID means a genuinely new post.

Q20. Can I schedule it to only run during certain hours?

Not built-in. However, you can use Windows Task Scheduler to start the program at a specific time and stop it at another time. The program itself always runs continuously once started.

SECTION 8 | Quick Reference Card

Commands

Command	What It Does
<code>python main.py</code>	Start monitoring + open dashboard at localhost:8080
<code>python main.py --no-gui</code>	Start in terminal-only mode (no browser)
<code>Ctrl + C in terminal</code>	Stop the program gracefully

Key Numbers

10	Users monitored simultaneously
45 sec	How often each user is checked (one full cycle)
3	Workers running in parallel
8 sec	Time to open and read one Twitter profile page
30–60 sec	Average time to detect a new post
50	Maximum recent detections shown in dashboard
150 MB	Approximate RAM usage
8080	Web dashboard port (localhost:8080)

Configuration File (config.py)

MONITORED_USERS	List of Twitter usernames to watch
CHECK_INTERVAL_SECONDS	Seconds between full cycles (default: 45)
WORKER_COUNT	Parallel workers (default: 3)
HEADLESS	True = no visible browser window (default: True)
NOTIFICATION_SOUND	True = play sound with notification (default: True)
DB_PATH	Location of the database file (default: data/monitor.db)

To add a new user to monitor: Open `config.py`, find the `MONITORED_USERS` list, add the username (without `@`) on a new line, save the file, and restart the program. Alternatively, use the Settings page at `localhost:8080`.